



HYTREX

CLARITY. PURPOSE. IMPACT.

DevUP

Propuesta de arquitectura

Por qué se siente como catorce cosas pegadas, y qué lo arregla

46 / 6

tablas de dominio, y seis
claves foráneas que cruzan dominios

0

columnas relacionales en la tabla
de tareas, fuera de tareas

0

llamadas a un modelo
en todo el repositorio

DIAGNÓSTICO SOBRE EL REPOSITORIO

46 tablas y 27 migraciones recorridas · auditoría de las
catorce pantallas probada a mano · 9 de septiembre de 2026

Resumen

DevUP funciona. Tiene canales de texto y voz cifrada, biblioteca de archivos, tablero de tareas, control de ventas, conector de GitHub, un entorno de desarrollo embebido, un mundo recorrible y catorce pantallas más. Nada de esto es una promesa: está desplegado y se usa.

Y aun así, usarlo no se siente como usar un producto. Se siente como abrir catorce herramientas que comparten una barra lateral. Este documento explica por qué —con las cifras del propio repositorio— y propone el cambio estructural que lo arregla, junto con el plan para que cada pantalla deje de ser superficial.

«El enemigo de este producto no es una empresa. Es la pérdida de contexto entre ventanas.»

Esa frase está en la landing desde el 2 de septiembre, y al lado tiene otra: **el estado se deduce de lo que pasó, no de lo que alguien anotó**. Las dos son la tesis correcta. Ninguna de las dos se cumple todavía, y el motivo no es falta de funciones: es que no existe el modelo de datos que las haría posibles.

Las tres cifras que resumen el diagnóstico

- **46 tablas de dominio y seis claves foráneas que cruzan dominios.** Todo lo demás apunta vertical a `organization_id` o a `user_id`. Los dominios no se conocen entre sí.
- **Cero columnas relacionales en la tabla de tareas,** fuera de tareas. Una tarea no puede apuntar a un commit, a un mensaje, a un cliente ni a un despliegue. El trabajo y aquello sobre lo que se trabaja viven separados.
- **Cero llamadas a un modelo en todo el repositorio.** Lo que hoy se presenta como agéntico son seis reglas condicionales cableadas y tres expresiones regulares sobre SQL.

La propuesta cabe en una frase: **introducir una columna vertebral —un nodo y un enlace— y colgar de ahí todo lo que ya existe.** No añade pantallas; les da un sitio común. Sobre esa columna se construyen las dos cosas nuevas que se han pedido: un espacio de notas enlazadas al estilo de Obsidian, y un motor agéntico —Hytrex Hearth— que no cueste dinero porque cada persona conecta su propio Claude por MCP.

Y hay una corrección importante respecto a la primera versión de esta propuesta. Infraestructura, Base de datos, Integraciones y Entorno de desarrollo no funcionan hoy, y la primera idea fue retirarlas de la barra y convertirlas en simples fuentes del grafo. **Eso ya no aplica: se están vendiendo como funciones.** Lo que se vende, se termina. La Parte III de este documento dice exactamente cómo.

El diagnóstico

1. El tejido actual es un array en la barra de navegación

Literalmente. Lo único que hoy relaciona Ventas con GitHub, con Noticias o con Infraestructura es que sus seis rutas están en la misma lista de la barra lateral. En el código no se conocen: esas seis pantallas no importan ni un componente la una de la otra, y sus únicos enlaces salientes van a github.com y a la web del proveedor.

No es una impresión. Es el resultado de recorrer las 27 migraciones y las catorce superficies buscando referencias cruzadas. Estos son los seis únicos cruces que existen en la base de datos:

Cruce	Migración	Qué une de verdad
archivos canales	0002:50	Un archivo recuerda de qué canal salió.
mensajes archivos	0005:25	Un mensaje puede llevar un archivo adjunto.
tareas archivos	0004:176	Solo comparten la tabla de etiquetas. Es taxonomía, no relación.
grabaciones archivos	0004:285	La grabación de una llamada se guarda como archivo.
mundo canales	0007:66	Una zona de DevVerse es un canal de voz. El cruce más fuerte del sistema.
infra y GitHub bóveda	0021:64 · 0016:22	Cuelgan de una credencial, no el uno del otro.

La mesa de trabajo ya había escrito el diagnóstico en un comentario, hace semanas: «hay veintiuna pantallas y todas valen lo mismo, una lista plana en una barra». Estaba bien visto. Lo que faltaba era la conclusión estructural que se sigue de ahí.

2. Lo que hoy no se puede hacer, y debería

Con esas seis relaciones en la mano, la lista de imposibles se escribe sola. Ninguna de estas cosas es un capricho: son las preguntas que un equipo se hace todos los días.

- **Un despliegue no sabe de qué commit salió.** Guarda el hash, el mensaje y el autor como texto plano, sin apuntar ni al repositorio ni a la persona. Despliegues y GitHub no se hablan en la base de datos.
- **Un repositorio no pertenece a ningún espacio de trabajo.** Cuelga de una credencial de la organización, así que no hay forma de decir «este repo es de este equipo».

- **Un cliente no tiene canal, ni carpeta, ni tablero.** No se puede abrir «la conversación del cliente X» ni ver qué tareas cuestan ese dinero.
- **Una tarea no se puede cerrar con un commit.** La relación más obvia de todo el producto —esto se hizo con este código— no existe.
- **El entorno de desarrollo no tiene ni una tabla.** Todo vive en el navegador: nada de lo que se escribe ahí se puede guardar, adjuntar ni convertir en tarea. Es un callejón sin salida.
- **No hay registro de actividad ni bus de eventos.** Una notificación no apunta a un objeto: apunta a una cadena de texto con una URL. Y solo se emiten cuatro tipos, sobre los cinco que la propia tabla declara.

Los eventos que hoy no producen nada

Cerrar o mover una oportunidad. Alcanzar un objetivo de ventas. Un despliegue fallido. Un push o una integración continua en rojo. Subir un archivo. Empezar una llamada. Mover una tarea de columna. Un vencimiento que llega. Nada de esto deja rastro en ningún sitio consultable — y es exactamente el material del que se deduce «en qué va» un proyecto.

3. La superficialidad, pantalla por pantalla

El 9 de septiembre se probó cada apartado a mano. El resultado no es cómodo, y por eso es útil: la mitad de las pantallas hacen bastante menos de lo que su sitio en el menú promete.

Pantalla	Estado	Qué hace hoy, exactamente
Panel	Funciona	Saludo, fecha, lo que te espera, quién está conectado, música, infraestructura y tareas. Es el único agregador real que existe.
Canales	Funciona	Texto, voz y vídeo. La pieza más sólida del producto.
Ventas	Funciona	Altas y bajas de clientes, ventas y servicios, con balance automático.
Mesa	A medias	Parte la pantalla en zonas, pero el catálogo de herramientas está incompleto.
Biblioteca	A medias	Sube archivos y los deja en el espacio. La persistencia real está sin verificar.
Tablero	A medias	Solo añadir tareas. Ni asignación con criterio, ni arrastre entre columnas.
GitHub	A medias	Solo visualiza commits. La conexión por token falla.
Noticias	A medias	Se publican y notifican, pero el detalle lleva fuera del espacio sin vuelta clara.
Infraestructura	No funciona	Tiene tablas de entornos y despliegues, y no las llena nadie.
Base de datos	No funciona	Solo lista las migraciones de un repositorio de GitHub.

Pantalla	Estado	Qué hace hoy, exactamente
Integraciones	No funciona	Seis reglas cableadas que diagnostican y no montan nada.
Entorno de desarrollo	No funciona	Cambia de pestaña al entrar, solo ofrece Node y no arranca en la VPS.
Ajustes	Básico	Foto, lista de personas, enlaces e invitaciones.
Perfil de usuario	No existe	No hay ninguna personalización de la persona.

Hay un patrón detrás de esta tabla, y conviene nombrarlo: **lo que funciona es lo que no necesita hablar con nada más**. Los canales se bastan solos. Ventas se basta sola. Lo que falla es todo aquello cuyo valor dependía de cruzar dominios — que es justamente lo que el producto promete.



La arquitectura

4. La columna vertebral: un nodo y un enlace

Un producto se siente como uno solo cuando tiene una columna vertebral. En Notion es el bloque; en Linear, la incidencia; en Obsidian, la nota enlazada. DevUP no tiene ninguna: el espacio de trabajo es un contenedor, no una columna.

La propuesta son **dos tablas**, no catorce pantallas nuevas.

Tabla	Qué guarda	Por qué así
nodos	Identidad de cualquier cosa: tipo, id de origen, título, organización.	No duplica el dato. Una tarea sigue viviendo en su tabla; el nodo solo le da una dirección estable en el grafo.
enlaces	Origen, destino, tipo de relación, procedencia y evidencia.	El tipo de relación importa: «cierra», «menciona», «pertenece a» y «desplegó» no son lo mismo, y de ahí sale la lectura.

Cada cosa que ya existe gana identidad de nodo sin cambiar su tabla: tarea, commit, cliente, venta, archivo, mensaje, despliegue, canal, migración, hallazgo. Y cualquier nodo puede enlazar con cualquier otro. Eso es todo el cambio.

La nota es un tipo de nodo, y la bitácora es una vista

Aquí es donde encaja lo que se pidió al estilo de Obsidian, sin construir un producto aparte. La nota en markdown es un nodo más —uno cuyo contenido lo escribe una persona en vez de deducirse de un evento— y los `[[enlaces]]` de Obsidian son, exactamente, filas en la tabla de enlaces.

Y la bitácora del proyecto no es una tercera tabla: es, **el mismo grafo ordenado por tiempo**. Preguntar «qué pasó esta semana» es recorrer los nodos por fecha; preguntar «de qué va esto» es recorrer sus vecinos. Un solo modelo, dos lecturas.

5. Procedencia: la firma de cada enlace

Se ha decidido que el sistema enlace solo, sin preguntar. Es lo que más tiempo ahorra y también lo que puede llenar el grafo de basura, así que la salvaguarda tiene que estar en el diseño y no en un botón de confirmación.

La salvaguarda es la **procedencia**: cada enlace guarda quién lo afirmó y con qué evidencia. Tres orígenes posibles, y se distinguen siempre.

Procedencia	Quién lo afirma	Qué se puede hacer con él
humano	Alguien lo escribió a mano.	Se respeta siempre. Nunca se borra en bloque.

Procedencia	Quién lo afirma	Qué se puede hacer con él
regla	Una regla determinista: el número de tarea en el mensaje del commit.	Auditable y reproducible. Si la regla estaba mal, se corrige y se recalcula.
agente	Un modelo lo propuso leyendo el contexto.	Se puede filtrar de la vista, revisar en lote y deshacer por completo.

Un grafo ruidoso con procedencia se filtra, se audita y se deshace. Un grafo ruidoso sin procedencia solo se puede borrar entero — y con él, lo que sí valía.

6. El registro de actividad, que hoy no existe

Para que el estado se deduzca de lo que pasó, hace falta que lo que pasa quede escrito. Hoy no queda: cuatro disparadores en 27 migraciones, y dos de ellos solo tocan una fecha de modificación. El tiempo real es un reparto en memoria que no persiste nada.

El registro de actividad es una tabla que anota, para cada evento de dominio, qué pasó, sobre qué nodo y por obra de quién. De ahí salen tres cosas de golpe: la bitácora, las notificaciones que por fin apuntan a un objeto en vez de a una URL, y el material que leen las reglas para tejer enlaces.

7. Jerarquía: tres niveles en vez de catorce iguales

La barra actual pone catorce destinos al mismo nivel, y el resultado es que ninguno destaca. La jerarquía los ordena por frecuencia de uso real, no por antigüedad de construcción.

Nivel	Qué contiene	Por qué ahí
1 · Columna	Hytrex Hearth: el grafo, las notas, la búsqueda y el agente.	Es el sitio al que se vuelve entre una cosa y otra. Sustituye al Panel como casa.
2 · Trabajo	Canales, Tablero, Biblioteca, Ventas y la Mesa.	Se tocan a diario. Sitio fijo y visible.
3 · Instrumentos	GitHub, Infraestructura, Base de datos, Integraciones y Entorno de desarrollo.	Se consultan cuando hacen falta, y además alimentan el grafo de forma continua. Siguen siendo pantallas completas: se venden como funciones.

El Hearth reemplaza al Panel. No conviven: el Panel de hoy es un agregador con cuatro fuentes cableadas a mano, y el Hearth es el mismo trabajo hecho sobre el grafo, sin límite de fuentes. Lo que el Panel hace bien —la lista de «te espera», la lectura de un vistazo— se conserva como una vista del grafo.

Corrección respecto a la primera versión

La primera versión de esta propuesta retiraba del menú las cuatro pantallas que no funcionan y las convertía en fuentes invisibles del grafo. Era una idea razonable con la información equivocada: **esas**

cuatro se están vendiendo como funciones del producto. Retirarlas de la vista sería vender algo que no se puede enseñar. Se quedan en el nivel 3, como pantallas completas, y la Parte III explica cómo terminarlas.

8. Hytrex Hearth: el grafo con una puerta MCP

Hearth es dos cosas a la vez. Por un lado, la **superficie** donde el grafo se ve y se escribe: el Obsidian propio, con notas en markdown, enlaces con autocompletado, backlinks, vista de grafo y la bitácora. Por otro, un **servidor MCP** que expone ese mismo grafo a un modelo.

Por qué el motor agéntico no cuesta dinero

Esta es la decisión que hace viable todo lo demás. DevUP **no compra inferencia**. No hay clave de API de un modelo en el servidor, ni cuota que se agote, ni factura que crezca con el uso. En su lugar, cada persona conecta su propio Claude —el que ya paga— y DevUP se presenta ante él como un servidor MCP.

Las consecuencias son mejores de lo que parece. El coste de inferencia es de quien lo usa, así que escala sin arruinar a nadie. El aislamiento sigue siendo el mismo de siempre, porque el servidor MCP habla con las credenciales de quien lo conecta y RLS sigue siendo la frontera. Y el motor no hay que construirlo: hay que **exponerlo**, que es un orden de magnitud menos de trabajo.

Qué expone el servidor, y con cuánto poder

Herramientas	Clase	Para qué sirve de verdad
buscar_nodos · leer_nodo · vecinos · bitacora	Lectura	«¿En qué va el cobro de Clínica Santa Ana?» — y contesta con la venta, sus mensajes, la tarea y el commit que la cerró.
crear_nota · enlazar · crear_tarea · mover_tarea	Escribe dentro	Convierte una conversación en tarea enlazada a su hilo. Todo con procedencia de agente y reversible en bloque.
abrir_pr · desplegar · editar_archivo	Actúa fuera	El nivel peligroso. Detrás de aprobación humana explícita, con registro de quién aprobó qué y cuándo.

9. Permisos: el grafo es del equipo

El grafo es **grupal**, no personal. Un nodo y un enlace pertenecen a la organización, con el mismo aislamiento por RLS que tiene todo lo demás — que en este caso importa más que nunca, porque nodos y enlaces tocan todos los dominios a la vez y un fallo ahí filtraría todo el producto de una vez.

Para lo que el agente hace fuera de DevUP —abrir una petición de cambio, desplegar, tocar código— la aprobación la da **el responsable dentro del rol del equipo**: quien responde de esa área, no cualquiera con permiso de administración. Eso pide una pieza que hoy no existe: un rol por área, con su responsable. Es poco trabajo y es la condición para que el nivel peligroso sea aceptable.

Profundizar lo que ya hay

Esta parte es la que faltaba. La columna vertebral hace que el producto se sienta como uno solo, pero no vuelve profundo lo que hoy es superficial. Cada apartado dice tres cosas: qué hace hoy, qué le falta para estar terminado, y qué gana cuando el grafo exista.

10. Tablero: de lista de tareas a mesa de reparto

Hoy solo añade tareas. Falta lo que convierte un tablero en una herramienta: **arrastrar entre columnas** con el orden persistido, asignación con criterio en vez de un desplegable, y subtareas o dependencias para el trabajo que no cabe en una fila.

Lo que gana con el grafo es lo importante: una tarea deja de ser una isla. Se cierra con el commit que la resolvió, cuelga del cliente que la pidió, arrastra la conversación donde se decidió y enseña el despliegue que la puso en producción. Y con el agente, «reparte esto entre el equipo» deja de ser una frase y pasa a ser una acción con contexto: quién ha tocado ese código antes, quién tiene menos carga, qué depende de qué.

11. Mesa: el catálogo son vistas del grafo

La mesa parte la pantalla en zonas y eso ya funciona. Lo que está incompleto es el catálogo: cinco herramientas de las catorce superficies, y las que son pantallas enteras —ventas, infraestructura— no entran porque no se pueden partir.

Con el grafo el catálogo se resuelve solo, porque **toda vista del grafo es una herramienta de mesa**: los vecinos de un nodo, una consulta guardada, la bitácora filtrada, un canal, un tablero. La mesa deja de necesitar que alguien escriba un widget por dominio.

12. Biblioteca: primero verificar, luego versionar

La biblioteca sube archivos y los deja en el espacio de trabajo, y hay una duda anotada que hay que despejar antes de añadir nada: **si los archivos persisten de verdad**. El almacén es MinIO en Railway con volumen, y hasta que alguien suba, cierre sesión, vuelva y descargue, eso es una suposición y no un hecho. Es la primera tarea del apartado y no cuesta casi nada.

Lo que le falta después: versiones —hoy subir dos veces el mismo nombre no tiene una respuesta clara—, vista previa sin descargar para lo que se puede previsualizar, y carpetas o algo que ordene cuando haya doscientos archivos en vez de veinte.

Con el grafo, un archivo cuelga de la tarea, del cliente o de la nota que lo justifica, en vez de vivir en una lista plana donde solo lo encuentra quien recuerda su nombre.

13. Ventas: de altas y bajas a embudo con memoria

Ventas es de lo mejor que hay: clientes, servicios, oportunidades, cotizaciones y balance automático. Y aun así es un archivador. Le falta el historial de cada cliente —qué se le dijo, cuándo y quién—, la cotización como documento que se pueda enviar, y avisos con antelación sobre los vencimientos en

vez de una fecha que hay que ir a mirar.

El grafo le da lo que un CRM no puede tener solo: el cliente enlazado a su canal de conversación, a las tareas que cuestan ese dinero y al despliegue que entregó lo prometido. «¿Qué le debemos a este cliente?» pasa de ser una pregunta de memoria a una consulta.

14. GitHub: arreglar el token y dejar de pedirlo

Dos problemas distintos, y el segundo es más interesante que el primero.

El primero es que **la conexión por token falla**. Hay que reproducirlo, encontrarlo y arreglarlo: es un fallo, no una función que falte.

El segundo es la propuesta que se planteó: que baste con **pegar el enlace del repositorio**. Para un repositorio público eso funciona sin credencial ninguna y elimina el paso que más gente pierde. Para uno privado no hay forma de evitar una autorización, pero la puede dar la aplicación de GitHub en vez de un token pegado a mano — que es más seguro y más cómodo que lo de hoy. Merece la pena: es el punto donde más gente se cae.

Y hay un techo que conviene decir en voz alta: la vista de commits es «solo lectura» por límites de la API, y eso es cierto para el historial completo, pero no para lo que de verdad hace falta. Ramas, peticiones de cambio, estado de la integración continua y quién revisa qué sí se pueden traer. El grafo los quiere a todos como nodos.

15. Noticias: unificar la vuelta

Se publican bien y notifican bien. El problema es de navegación: pulsar para ver el detalle lleva a una pantalla distinta del espacio de trabajo, y volver no es intuitivo. Carlos ya corrigió una parte —el enlace del widget de la mesa respeta el contexto desde el 8 de septiembre— y falta el resto: que el detalle viva dentro del espacio y que «volver» devuelva a la pestaña donde se estaba, no a un sitio fijo.

Es el ejemplo más pequeño y más claro de todo el documento: la función estaba, y lo que fallaba era el tejido.

16. Infraestructura: se vende, hay que terminarla

Las tablas de entornos y despliegues ya existen, con su credencial asociada en la bóveda. Lo que no existe es lo que las llena: **nadie pregunta al proveedor**. Terminarla es leer de Railway, de Cloudflare o de quien sea, con la credencial que ya está guardada, y escribir los despliegues que vuelven.

Con eso, la pantalla enseña lo que promete —qué hay en pie, qué se desplegó, qué falló— y de paso alimenta el grafo con el nodo que más conecta de todos: un despliegue que apunta a su commit, y por él a la tarea y a la persona.

17. Base de datos: de listar migraciones a gobernarlas

Hoy lista las migraciones de un repositorio y las analiza con tres reglas regulares que marcan lo destructivo. El análisis es útil y está bien pensado; lo que falta es todo lo que se hace con él: **saber qué migración está aplicada en qué entorno**, aplicarla desde ahí, y tener escrito el camino de vuelta antes de aplicarla.

Este apartado tiene una ventaja que ningún otro tiene: el criterio ya existe y ya está probado en el propio repositorio. La lección de que `db:migrate` cambia la contraseña de la aplicación, y el diseño de expansión y contracción para lo que puede perder datos, están escritos y costaron caro. Esta pantalla es donde ese criterio se convierte en producto.

18. Integraciones: lo que nadie más hace

Seis reglas cableadas que leen el código de un repositorio y dicen qué se ahorraría con qué herramienta, cada una con su evidencia en archivo y línea. Y el propio código es honesto sobre su límite: «lo que todavía no hace, y conviene no fingirlo: montar la integración».

Aquí está el apartado entero. Diagnosticar es la mitad barata; **montar es la mitad que nadie más hace**. Crear el proyecto en el proveedor, guardar las claves en la bóveda que ya existe y escribir el esquema inicial es exactamente el trabajo que el producto promete ahorrar, hecho de verdad.

Y es el apartado que más gana con el agente por MCP, porque el diagnóstico deja de depender de seis reglas escritas a mano: con el repositorio expuesto como nodos, un modelo lee lo que hay y encuentra lo que ninguna expresión regular iba a encontrar.

19. Entorno de desarrollo: tres problemas separados

Es el apartado en peor estado y conviene separar sus problemas, porque tienen dificultades muy distintas.

- **Cambia de pestaña al entrar.** Es un fallo de diseño heredado de una restricción real: la página necesita cabeceras de aislamiento para arrancar, así que se sirve con una navegación dura. Se puede resolver sin renunciar a las cabeceras, y hay que hacerlo: hoy parece que la aplicación se rompe.
- **Solo ofrece Node.** Es lo que permite el navegador con WebAssembly. Ampliarlo de verdad significa ejecutar fuera del navegador, y eso ya no es la misma función: es una decisión de producto y de coste.
- **No arranca en la VPS.** Hay que averiguar por qué antes de prometer nada — casi con seguridad son las cabeceras de aislamiento otra vez, que dependen de cómo sirva el proxy.

Y sobre todo: **cerro tablas**. Mientras nada de lo que se escribe ahí se pueda guardar, el apartado es una demostración y no una herramienta. Persistir el proyecto es lo primero, y con el grafo pasa a poder colgar de una tarea o de una nota.

20. Ajustes y perfil: la persona no existe todavía

Los ajustes de la organización dejan cambiar la foto, ver a las personas, añadir enlaces e invitar. Falta lo que se echa en falta en cuanto hay más de tres personas: roles de verdad —con el responsable por área que pide el apartado 9—, permisos por espacio de trabajo y un registro de quién hizo qué.

Y del **perfil de la persona no hay nada**. Ni foto, ni nombre visible, ni a qué se dedica, ni preferencias. En un producto donde el estado de presencia sí existe y se ve, y donde hay un mundo recorrible con avatares, eso es un hueco raro. Es además el apartado más pequeño de esta parte y el que más se nota.

21. Lo que se queda como está

Canales es la pieza más sólida del producto: texto, voz y video cifrados, y no le hace falta nada estructural. **DevVerse** no es un nivel de la jerarquía sino una vista alternativa de todo lo demás, y conviene que siga siendo opcional. **Música** —Spotify y YouTube— está terminada dentro de lo que permiten sus proveedores.

Nombrar lo que no se toca es parte del plan: sin eso, un documento como este parece decir que todo está mal.



El orden y los riesgos

22. Seis bloques, y por qué en ese orden

Se ha pedido todo, y todo se puede hacer. Lo que no se puede es a la vez. Este orden tiene una regla detrás: **cada bloque se despliega solo y demuestra algo antes de que empiece el siguiente**. El primero no construye ni una función nueva.

	Bloque	Qué demuestra
1	La jerarquía y las vueltas	Reordenar la barra en tres niveles y arreglar la navegación de Noticias. Es mover enlaces, no escribir funciones — y demuestra que la sensación de «catorce cosas pegadas» era jerarquía, no falta de producto.
2	Nodos, enlaces y actividad	Las dos tablas con RLS y su caso de aislamiento en el mismo commit, más el registro de actividad. De paso se arreglan las tres deudas que el diagnóstico dejó a la vista: el despliegue apunta a su commit, el repositorio a su espacio, la notificación a un objeto.
3	El tejido determinista	Reglas que enlacen sin modelo: commit a tarea por el número, despliegue a commit, mensaje a cliente. Gratis, auditable y no alucina. Demuestra «el estado se deduce de lo que pasó» con evidencia.
4	Hearth como superficie	El Obsidian propio: notas, enlaces, backlinks, grafo y bitácora, sobre un grafo que ya no nace vacío. Y sustituye al Panel.
5	La puerta MCP	Primero solo lectura, luego escritura dentro de DevUP, y lo que actúa fuera al final con aprobación del responsable. Demuestra el ahorro real: preguntarle al proyecto en vez de reconstruir el contexto.
6	Terminar los instrumentos	Infraestructura, Base de datos, Integraciones y Entorno de desarrollo, que se venden y hay que entregar. Van después del grafo a propósito: así nacen conectados en vez de ser cuatro islas más.

El bloque 6 podría ir antes, y hay un argumento para ello: se está vendiendo. El argumento en contra es más fuerte: terminarlos antes del grafo significa escribirlos dos veces, porque lo que los vuelve valiosos es precisamente estar conectados. Si la presión comercial obliga, el orden se cambia — pero sabiendo lo que cuesta.

23. Cuatro cosas que pueden salir mal

RLS falla en silencio

Una tabla sin política devuelve cero filas y ningún error. Nodos y enlaces tocan todos los dominios a la vez, así que un fallo ahí no filtra un dominio: filtra el producto entero. Política y caso en las pruebas de aislamiento en el mismo commit que la tabla, sin excepción.

El grafo se llena de ruido

Enlazar automático sin procedencia produce un grafo que nadie se cree y que no se puede limpiar. La procedencia no es un adorno del diseño: es la condición para que el enlace automático sea aceptable.

El agente que actúa fuera

Abrir peticiones de cambio y desplegar con las credenciales de la organización es el poder más útil del plan y el más peligroso. Va al final, detrás de la aprobación del responsable del área, y con todo lo que hace registrado como nodo — para que se pueda auditar y deshacer.

El grafo grupal enseña más de lo que parece

Si todo se enlaza con todo y el grafo es del equipo, una nota personal sobre una venta la ve quien tenga acceso a esa venta. Eso es lo correcto para un producto de equipo, y hay que decirlo antes de que alguien escriba algo pensando que era privado.

24. Lo que decide quien manda

- **El orden del bloque 6.** Terminar los instrumentos antes o después del grafo: más rápido de enseñar, o escrito una sola vez.
- **Qué es un «responsable de área».** El apartado 9 lo necesita para que el agente pueda actuar fuera, y hoy solo hay dueño, administrador y miembro.
- **Ampliar el entorno de desarrollo fuera del navegador.** Es la única pieza del plan que cuesta dinero de servidor, y por eso es una decisión y no una tarea.
- **Qué se le promete al cliente mientras tanto.** Cuatro pantallas se venden hoy y estarán terminadas en el bloque 6. Eso es una conversación comercial, no técnica.

Todo lo demás de este documento es trabajo, y el trabajo se puede empezar mañana por el bloque 1, que no rompe nada y ya se nota.